Lecture 23: SQL DML

INSERT, UPDATE, and DELETE

Z2004: Database Management Systems

D0.jpg

IIT Madras Zanzibar

24 April 2026

What is DML?

DML (Data Manipulation Language) is the part of SQL used to **change the data** stored in tables.

Core DML commands:

- `INSERT` — add new rows
- `UPDATE` — modify existing rows
- `DELETE` — remove rows

Note: `SELECT` reads data (often called DQL), not DML.

DML Syntax Patterns

INSERT (new row):

```
INSERT INTO TableName (col1, col2, ...)
VALUES (val1, val2, ...);
```

UPDATE (change rows):

```
UPDATE TableName
SET    col1 = newValue,
       col2 = expression
WHERE  condition;
```

DELETE (remove rows):

```
DELETE FROM TableName
WHERE  condition;
```

Workflow: write `WHERE` first → run as `SELECT` → then run `UPDATE/DELETE`.

INSERT (Patterns) — Common INSERT Forms

```sql
-- Insert one row
INSERT INTO table_name (column1, column2)
VALUES (value1, value2);

-- Insert multiple rows
INSERT INTO table_name (column1, column2) VALUES
(value1a, value2a),
(value1b, value2b),
(value1c, value2c);

-- Insert from another table
INSERT INTO table2
SELECT * FROM table1 WHERE condition;
```

INSERT INTO — Adding Rows

```sql
USE SchoolDB;

-- 1) Insert a student (SID is IDENTITY, so we omit it)
INSERT INTO Student (Name, Dept, GPA)
VALUES (N'Asha Ali', 'CSE  ', 3.80);

-- 2) Insert a course (natural key CID)
INSERT INTO Course (CID, CName, Credits)
VALUES ('DB101', N'Introduction to Databases', 3);

-- 3) Enrol the student (FKs must match existing Student and Course)
INSERT INTO Enrolment (SID, CID, Grade)
VALUES (1, 'DB101', NULL);
```

Tip: Always name columns in `INSERT`.

UPDATE — Modifying Existing Rows

```sql
USE SchoolDB;

-- 1) Update one student's GPA
UPDATE Student
SET    GPA = 3.90
WHERE  SID = 1;

-- 2) Update multiple columns at once
UPDATE Student
SET    Dept = 'CSE  ',
       GPA  = GPA + 0.10
WHERE  SID = 1;

-- Preview rows BEFORE updating
SELECT SID, Name, Dept, GPA
FROM   Student
WHERE  SID = 1;
```

Rule: write `WHERE` first, test it as `SELECT`, then run `UPDATE`.

DELETE — Removing Rows

```sql
USE SchoolDB;

-- Preview what you are about to delete
SELECT SID, Name, Dept
FROM   Student
WHERE  Dept = 'CSE  ';

-- Delete rows by condition
DELETE FROM Student
WHERE  Dept = 'CSE  ';

-- Delete child rows (Enrolments) for a given course
DELETE FROM Enrolment
WHERE  CID = 'DB101';
```

Warning: Never run `DELETE` without `WHERE`.

DML Commands Compared — DELETE vs TRUNCATE vs DROP

Dimension	DELETE	TRUNCATE	DROP
Removes rows	Yes (selective)	All rows	All rows
Removes structure	No	No	Yes
WHERE clause	Yes	No	No
Triggers fire	Yes	No	No
Can rollback	Yes (if in txn)	Yes (logged)	No*
Resets IDENTITY	No	Yes	Removed
Speed	Slow (row-by-row)	Fast	Instant
Referential checks	Yes	Yes (FKs block)	Yes

*Only if inside an explicit transaction block

Common DML Mistakes

Mistake 1: UPDATE without WHERE

```
-- DANGER: updates every row!
UPDATE Student
SET    Dept = 'CSE  ';

-- Safer pattern
UPDATE Student
SET    Dept = 'CSE  '
WHERE  SID = 1;
```

Mistake 2: DELETE without WHERE

```
-- DANGER: empties the table!
DELETE FROM Enrolment;

-- Correct
DELETE FROM Enrolment
WHERE  SID = 1 AND CID = 'DB101';
```

Mistake 3: Foreign key violation

```
-- Fails if Student (SID=999) doesn
   't exist
INSERT INTO Enrolment (SID, CID,
   Grade)
VALUES (999, 'DB101', 'A');

-- Fix: insert the parent first
INSERT INTO Student (Name, Dept,
   GPA)
VALUES (N'Test', 'CSE  ', 3.00);
```

Best practice: test your WHERE as a SELECT first.

Summary

Key takeaways (L23: SQL DML)

- `INSERT` adds new rows; prefer naming the column list explicitly
- `UPDATE` modifies existing rows; always include a correct `WHERE`
- `DELETE` removes rows; always include a correct `WHERE`
- Safe workflow: test `WHERE` as a `SELECT` first, then run `UPDATE`/`DELETE`